

FORM 5 — AI Appendix — Case Study Attachment

i *Mandatory attachment for all case study submissions. To be completed by the candidate. Please be concise but specific. We evaluate your critical thinking, prompt strategy, and ability to refine AI outputs*

1. Tool Manifest

List all AI models and tools used. Please specify versions where applicable, e.g., Gemini Advanced, ChatGPT-4o, Midjourney v6.

Model / Tool	Primary Purpose of Use	Effectiveness (1-5) & Brief Why
Claude Opus 5.5 (Chat + Claude Code)	Plan, Code, Review	4.8 – I've used it since launch and it approaches problems the way I do. It also asked me questions instead of guessing when something was unclear.
GPT 5.6 Tera	Writing the first prompt, reviewing the plan	4 – Good for breaking the work into steps and pushing back on the plan, but Opus was better at the actual design and code

2. AI Tool Usage Approach

Detail your workflow. Do not paste full raw outputs; summarize your strategy and how you guided the AI.

Phase / Specific Task	Prompting Strategy & Context Provided	Human Refinement (How did you edit or iterate?)
Research	Gave GPT my background, the job description and the case study, and asked it to help me write a good prompt for planning	Made sure the prompt asked for analysis before any code, called out the tricky parts of each scenario, and asked what an interviewer would challenge.
Planning	Ran the brainstorming skill on AGENTS.md in Claude Code. It went through the plan against the case study and asked me questions before any code was written	I made the calls on every open question (overlapping promotions, rounding, pagination, etc.). Claude updated AGENTS.md with them and they're written down in the ADR.
Review and decisions	Gave Claude Code the case and the plan, and asked it to review the plan critically and ask me questions before writing code.	I made the calls on every open question (overlapping promotions, rounding, pagination, etc.) and wrote them down in the ADR.
Build and test	Built it step by step, then ran the code-review skill on the finished project.	Every fix needed a test. For tricky concurrency bugs, I removed the fix and made sure the test actually failed without it.
Manual testing	Took the workflows from the design doc and tried them myself	Ran the tests, called the API step by step (sales, product promos, new products during a sale, cancel, CSV upload with bad rows) and checked

		the database to make sure it matched what the API returned
Making it easy to run	Asked Claude for an OpenAPI spec with an interactive docs page, and a script that sets up and starts the whole project with one command.	Tried it myself end to end, so a reviewer can run npm run up and test everything from the browser without any setup

3. Judgement, Challenges & Verification

Highlight your critical thinking. How did you handle the limitations of the AI?

Challenge Encountered (e.g., hallucinations)	Judgement / Verification (How did you fact-check?)	Resolution (How did you fix the issue?)
The plan looked right but had a big hole: prices only updated when someone changed a promotion, not when a sale's end date passed. A flash sale would never end, and a scheduled one would never start.	Reviewed the plan against the case study. Then wrote a test with a fake clock that moves time forward past the sale's end, and tested the flows by hand.	Status is now calculated from the dates, each price knows when it expires, and a background job refreshes expired prices. The busiest endpoint also re-checks before it answers.
Tests and fixes that only looked right. One concurrency test passed even with the fix removed, and a later lock fix caused a deadlock with a different part of the system.	Removed each fix and re-ran its test to see whether it failed. The code-review skill caught the deadlock by checking lock order across the code.	Rewrote the tests so they reliably fail without the fix (one now shows a real Postgres deadlock error), then fixed the lock order.

4. Overall Reflection

Estimated Ratio: *What percentage of the final case study would you estimate is directly AI-generated vs. human-crafted/heavily edited? (e.g., 30% AI / 70% Human)*

95% AI / 5% Human. The AI wrote almost all the code and docs. My part was making the decisions, pointing the reviews in the right direction, testing the flows by hand against the database and not accepting a fix until I'd seen its test fail without it.
--

Key Takeaway: *Did using AI for this specific case study reveal any notable blind spots in the tools, or significantly alter your original approach? Briefly explain.*

I'd already sketched my own architecture before using AI, and the AI's plan was pretty close to it, so I trusted it and didn't question it much. That's why the gaps were easy to miss. The plan didn't handle sales ending on their date or two writes hitting the same product at once, and I didn't catch that by reading it. The review in Claude Code did. So AI didn't change my architecture much, but it changed how I check things: now I ask "what breaks when time passes, when two things happen at once, or when a message gets lost?" and I only trust a fix once I've seen its test fail without it.
